Ders 10 Genişletilmiş Notu: Ağaçlara Giriş, İkili Arama Ağaçları, Ağaçta Gezinme, Huffman Kodlaması, DFS, BFS ve Prim Algoritması

Ders 10 Genişletilmiş Notu: Ağaçlara Giriş, İkili Arama Ağaçları, Ağaçta Gezinme, Huffman Kodlaması, DFS, BFS ve Prim Algoritması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ağaç Nedir? Ağaç ile Graf Farkı
- 2. Köklü Ağaç, Seviye ve Yükseklik
- 3. Ağaç Terminolojisi
- 4. İkili Ağaçlar
- 5. İkili Arama Ağacı ve Dengeli Ağaç
- 6. Sıralı Diziden Dengeli İkili Arama Ağacı Kurmak
- 7. İkili Ağacın Dizide Tutulması
- 8. Ağaçta Gezinme (Traversal)
- 9. Huffman Kodlaması
- 10. Kapsayan Ağaçlar (Spanning Tree)
- 11. Derin Öncelikli Arama (DFS)
- 12. Geniş Öncelikli Arama (BFS)
- 13. En Küçük Kapsayan Ağaç ve Prim Algoritması
- Örnek Sorular
- Dersin Toparlaması

1. Ağaç Nedir? Ağaç ile Graf Farkı

Ders, ağaçlar (trees) konusuyla yeni bir bölüme geçti. Hoca, ağaçların birinci sınıftaki derslerden kısmen bilindiğini ama “ağaç deyip geçmeyelim, derin bir konudur” diye uyardı. Veri yapılarında son ek ağaçları (suffix tree), B-ağaçları (B-tree), trie gibi çok farklı ağaç modelleri vardır. Bunlar bu dersin içeriğinde yoktur. Bu derste daha temel yapılar anlatılır. Ağaç her zaman “tepetaklak” düşünülecek: kök (root) yukarıda, dallar üzüm salkımı gibi aşağı sarkar.

Hocanın ilk cümlesi şuydu: **Her ağaç bir graftır, ama her graf bir ağaç değildir.** Graf (graph) düğümlerden (vertex) ve kenarlardan (edge) oluşur. Grafta kapalı bir döngü (cycle) olabilir, yani başlanan düğüme geri dönülebilir. Grafta kapalı döngü yoksa bu graf aynı zamanda bir ağaçtır. Bu yüzden ağaçlar grafların bir alt modelidir.

Ağacın temel özelliği şudur: Bir düğümden başka bir düğüme giden **tek bir yol** vardır. Hoca slayttaki ağaçta v düğümünden w düğümüne yalnız siyahla çizilmiş yoldan gidilebildiğini, başka yol olmadığını gösterdi. Grafta ise kapalı döngüler yüzünden bir düğüme çok farklı yollardan gidilebilir. Hocaya göre ağaçlarda ve graflarda farklı algoritmalar kullanılmasının sebebi budur.

Hoca soldaki ağacın vücuttaki damarlara benzediğini söyledi. Ayrıca bir uyarı yaptı: “Ağaç” deyince yalnız ikili ağaç (binary tree) ya da bir kökten çıkan bir sürü dal düşünülmemeli. Tek

bir çizgi (iki düğüm ve bir kenar) da bir ağaçtır.

Ders sırasında ağacın denk tanımları da slaytta geçti: n düğümlü bir T ağacı bağlantılıdır ve $n-1$ kenarı vardır; ya da döngüsüzdür (acyclic) ve $n-1$ kenarı vardır. Hoca bu slaytı "önemli değil" diyerek hızla geçti. Döngüsüz (acyclic) graf kavramını da hatırlattı: G grafında döngü yoksa bu graf bir ağaçtır.

> **Ders sonrası not:** Hoca "grafta kapalı döngü yoksa bu aynı zamanda bir ağaçtır" dedi. Ağaç tanımında döngüsüzlüğün yanında bağlı (connected) olma şartı da vardır. Bağlı olmayan döngüsüz graf ağaç değil, ormandır (forest). Slayttaki denk tanımlarda "bağlantılı" kelimesi bu yüzden geçer.

2. Köklü Ağaç, Seviye ve Yükseklik

2.1 Köklü ve köksüz ağaç

Ağaçlar köklü (rooted) ya da köksüz olabilir. Köklü olması için hangi düğümün kök olduğu bilgisinin verilmesi gerekir.

> **Hocanın vurgusu:** Bu bilgi size verilmediği müddetçe ağaçtaki hangi düğümün kök olduğunu bilemezsiniz. Alışkanlıkla en tepedeki düğüme kök deriz, ama söylenmedikçe onun kök olduğunu iddia edemeyiz.

Hoca bunu infix ve postfix gösterimlerini çizdiği önceki derse bağladı. Orada en üstteki düğüm hep kök alınmıştı, ama aslında bu bilginin verilmesi gerekir. İstisnai bir durum yoksa ya da durum çok barizse en tepedeki düğüm kök kabul edilir.

Köksüz bir ağacın herhangi bir düğümü kök seçilebilir. Bu yalnız bir varsayımdır. Hocanın benzetmesi: Seçilen düğümü en yukarı alıp ağacı üzüm salkımı gibi kökten tutup aşağı doğru silkelerseniz ağaç yeni haline gelir. Slaytta 1 numaralı düğüm kök seçilip ağaç bu şekilde yeniden çizildi.

2.2 Seviye (level)

Bir düğümün seviyesi, o düğümden köke olan mesafedir (geçilen kenar sayısı). Kökün köke mesafesi sıfırdır, bu yüzden kökün seviyesi $L = 0$'dır. Köke bir kenarla bağlı düğümlerin seviyesi 1'dir. Köke ulaşmak için iki kenar geçilen düğümlerin seviyesi 2'dir. Slayttaki ağaçta en alttaki düğümün seviyesi 3'tü. Aynı seviyedeki dalların boylarının aynı çizilmesi gerekmez.

2.3 Yükseklik (height)

Bir ağacın yüksekliği, seviyelerin en büyüğüdür. Slayttaki ağaçta en yüksek seviye 3 olduğu için yükseklik 3'tür. Her düğümün seviyesini tek tek ölçmek yerine, en dıştaki yaprak ile kök arasındaki kenar sayısına bakmak da yeterlidir. Hoca bu ağaçta en dış yapraktan köke 1, 2, 3 kenar saydı.

3. Ağaç Terminolojisi

Hoca terminolojiyi aile ağacı benzetmesiyle anlattı. Aile ağacında hatırlanabilen en büyük dede köke yerleştirilir. Onun çocukları, çocuklarının evlendikleri ve onların çocukları aşağı doğru günümüze kadar gelir. Bu yüzden düğümler bir aile gibi adlandırılabilir. Köke root yerine ebeveyn (parent), yapraklara (leaf) çocuk (child) denebilir. Değişen bir şey yoktur, yalnız iki farklı terminoloji vardır.

- **Ebeveyn (parent):** Kendisinden dal çıkan, yani çocukları olan üst düğüm.
- **Çocuk (child):** Bir ebeveynden çıkan dallardaki düğümler.
- **Kardeş (sibling):** Aynı ebeveynden çıkan çocuklar. Hoca "aynı anne babadan olan çocuklar kardeştir" dedi.
- **Ata (ancestor):** Ata yalnız en tepedeki kişi değildir. Bir düğümün üstünde yer alan bütün düğümler onun atasıdır: babam, dedem, dedemin babası, onun babası.
- **Torun (descendant):** Bir düğümden sonra "dünyaya gelmiş" herkes, kendi çocukları dahil.

- **Uç düğüm (terminal vertex):** Ağacın en dışında, kendisinden dal çıkmayan düğüm. Yaprak da denir.
- **Ara düğüm (internal vertex):** Kendisinden dal çıkan her düğüm. İster ebeveyn, ister kök, ister ata olsun fark etmez. Kök de ara düğümdür.
- **Alt ağaç (subtree):** Bir ağacın içinden çıkarılan alt dallar.

Alt ağacı anlatmak için hoca alt dizgi (substring) benzetmesi yaptı. Bir kelimenin içinden "ba", "an", "nu", "anu" gibi alt dizgiler çıkarılabildiği gibi, bir ağacın içinden de alt dallar çıkarılabilir. Tek bir çizgi bile alt ağaç olabilir. Bir düğümden diğerine bağlantı varsa, tek yol varsa ve kapalı döngü yoksa bu bir ağaçtır.

Slaytta T ağacının düğüm kümesi 15 düğümden oluşuyordu. Koyu siyahla çizilen alt ağaç T' ise 5 düğümlüydü. T''nin düğüm kümesi T'nin düğüm kümesinin, kenar kümesi de T'nin kenar kümesinin alt kümesidir. Hoca, gösterimde V'nin düğüm (vertex), E'nin kenar (edge) kümesi olduğunu hatırlattı. Ara düğüm slaytlarda "int" diye kısaltılmıştı. Bu "integer" değil, "internal" demektir.

4. İkili Ağaçlar

4.1 İkili ağaç (binary tree)

İkili ağaçta bir düğümden **en fazla iki** kenar çıkar. Bir düğümün 0, 1 ya da 2 çocuğu olabilir. İkiden fazla kenar çıkan bir düğüm varsa ağaç ikili ağaç değildir. "İkili" adı da buradan gelir. Hoca slaytta tek çocuklu, çocuksuz ve iki çocuklu düğümler içeren birçok şekil gösterdi. Hepsi ikili ağaçtı. Bunların dengeli olup olmadığı ayrıca tartışılır.

4.2 Slayttaki ikili ağaç örneği

Hoca bir ağaç üzerinde sorular sordu ve cevapladı:

- Kökü verilmiş: A.
- A'nın iki çocuğu var. Sol çocuk B, sağ çocuk C.
- D'nin sağ çocuğu yok.
- Ara düğümler: A, C, D, E, H.
- Uç düğümler: B, F, G, I.
- Yükseklik: en dıştaki düğümle kök arasında 1, 2, 3, 4 kenar var. Yükseklik 4'tür.

Bu ağaçtan kaç alt ağaç çıkarılabileceğine dair bir formül yoktur. Hoca birkaç farklı alt ağaç gösterdi ve "olabilecek neler görüyorsam onlar" dedi.

4.3 İfade ağaçları

Matematiksel ifadelerin gösteriminde ikili ağaçlar kullanılır. Hoca önceki derste görülen infix gösterimini hatırlattı: sol, düğüm, sağ (left, node, right) diye okunur. Örneğin $A - B$.

- $A - B/C$: Önce B/C yapılır, sonuç A'dan çıkarılır.
- $(A - B) \cdot C$: Önce A'dan B çıkarılır, sonuç C ile çarpılır.

Hocaya göre ifadeye bakarak ağacın yüksekliği söylenebilir. Tek işlem varsa yükseklik 1, iç içe iki işlem varsa yükseklik 2'dir. İfade $(A-B)\cdot C\cdot(A+C)$ yapılırsa iç içe üç işlem olur, yükseklik 3'e çıkar. Arada bir yerde kalan ($A + C$ gibi) işlem ağacın yüksekliğine ayrıca katılmaz.

4.4 Aynı düğüm sayısı, farklı ağaçlar

Slaytta üç ağacın her birinde 7 düğüm ve 6 kenar vardı. Ama ağaçlar birbirinden çok farklıydı:

- Birincisi hem ikili hem tam ikili ağaçtı (hiç eksik dalı yok). Aynı zamanda dengeliydi: kökün sağındaki ve solundaki düğüm sayıları aynı. Yüksekliği 2.
- İkincisi ikili ama dengesiz bir ağaçtı. Yüksekliği 4.
- Üçüncüsü "garip kenar süslerine" benziyordu. Birçok dal boştu ama yine her düğümden en fazla iki kenar çıktığı için ikili ağaçtı. Yüksekliği 6.

Sonuç: Düğüm sayısının aynı olması, ağaçların aynı tipte olacağını ya da özelliklerinin aynı olacağını söylemez.

4.5 n düğümle kaç farklı ikili ağaç çizilir?

Hoca şöyle saydı:

- 0 düğüm: "Ağaç çizemem" dedi.
- 1 düğüm: 1 ağaç. Tek bir düğüm hem ağaç hem graftır. Kenar çıkmaması ağaç olmadığı anlamına gelmez.
- 2 düğüm: 2 ağaç. Biri sola, biri sağa dallanmış.
- 3 düğüm: 5 ağaç. Hoca birini, onun simetriğini ve "klasik kiraz modelini" (kök ve iki çocuk) gösterdi.

  Ders sonrası not: Hoca "0 düğümle ağaç çizemem" dedi. Ağaç sayımında boş ağaç da bir ağaç sayılır ve 0 düğüm için sayı 1 alınır. 1, 2 ve 3 düğüm için hocanın verdiği 1, 2, 5 sayıları doğrudur.

  Ek bilgi (derste anlatılmadı): Bu sayılar Catalan sayılarıdır: $C_n = \frac{1}{n+1}\binom{2n}{n}$. $n = 0, 1, 2, 3, 4$ için 1, 1, 2, 5, 14 çıkar. Değerler Python ile kontrol edildi.

4.6 Tam ikili ağaç (full binary tree)

Hoca önce "her düğümden en fazla iki kenar çıkıyorsa full binary'dir" dedi, sonra tanımı düzeltti: **Uç düğümler hariç her düğümün tam iki çocuğu vardır.** Slayttaki ağacı tam yapmak için eksik dallara mavi düğümler ekledi.

Slayttaki özellikler:

- t uç düğümlü bir ağaçta yükseklik için $\lceil \log_2 t \rceil \leq h$ geçerlidir.
  - Siyah ağaçta 5 uç düğüm vardı. $\log_2 5 \approx 2{,}32$, yükseklik 3. $2{,}32 \leq 3$, sağlanıyor.
  - Mavilerle tamamlanmış ağaçta 8 uç düğüm vardı. $\log_2 8 = 3$, yükseklik 3. Eşitlik sağlanıyor.
- i ara düğüm varsa $i + 1$ uç düğüm vardır.
  - Siyah ağaçta 4 ara düğüm, 5 uç düğüm.
  - Mavilerle tamamlanmış ağaçta 7 ara düğüm, 8 uç düğüm.
- i ara düğüm varsa toplam düğüm sayısı $2i + 1$'dir.
  - Siyah ağaçta $2 \cdot 4 + 1 = 9$ düğüm. Hoca 1'den 9'a kadar saydı.

Soru: "i adet ara düğüm" diyor, orada i adet uç düğüm olması gerekmiyor mu? Slayttaki örnek öyle vermiş gibi.

Cevap: Alttaki örnekle karıştırmayın. Siyah ağaçta 4 ara düğüm var, formül toplam $2 \cdot 4 + 1 = 9$ düğüm diyor ve gerçekten 9 düğüm var. Slayttaki örnekte de A, B, C, D ara düğüm (4 tane), geri kalan 5 düğüm uç düğüm, toplam 9. Formülde bir problem yok.

> **Ders sonrası not:** Derste üç ifade düzeltilmelidir. - Hoca önce "her düğümden en fazla iki kenar çıkıyorsa full binary'dir" dedi. Bu ikili ağacın tanımıdır. Tam ikili ağaçta uç düğümler dışındaki her düğümün **tam** iki çocuğu vardır; hoca bunu hemen sonra doğru söyledi. - Hoca $\lceil \log_2 t \rceil \leq h$ eşitsizliğinin "yalnız full binary ağaçlar için geçerli, her binary tree için değil" olduğunu söyledi. Bu eşitsizlik her ikili ağaçta geçerlidir, çünkü h yükseklikli bir ikili ağaçta en fazla 2^h yaprak olabilir. - Hoca "full binary ağaçta uç düğüm sayısının logaritması yüksekliğe eşit olur" dedi. Eşitlik yalnız bütün yaprakların aynı seviyede olduğu (mükemmel) ağaçta, örneğin 8 yapraklı mavi ağaçta geçerlidir. 5 yapraklı siyah ağaç tam ikili ağaçtır ama $\log_2 5 \neq 3$.

4.7 Seviyedeki en fazla düğüm sayısı

İkili ağaçta l. seviyede en fazla 2^l düğüm bulunabilir. Seviye 0'da $2^0 = 1$ düğüm (kök), seviye 1'de $2^1 = 2$, seviye 2'de $2^2 = 4$, seviye 3'te $2^3 = 8$ düğüm olabilir. Hoca bunu mavilerle tamamlanmış ağaçta saydı.

5. İkili Arama Ağacı ve Dengeli Ağaç

5.1 Kelimelerle ikili arama ağacı

Hoca, önceki hafta sıralı bir sayı dizisinin ikili arama ağacına (binary search tree, BST) yerleştirildiğini hatırlattı. Orada ağacın hep dengeli olması istenmişti. Sebebi şudur: İkili aramada hep ortadan bölündüğü için, aranan sayıya aynı sayıda soruyla ulaşmak isteniyorsa bölünen yerin solunda ve sağında aynı sayıda eleman olmalıdır.

Yeni örnekte bir cümle verildi: “computers are an important technological tool”. Kelimeler, aranırken ikili arama mantığı kullanılabilecek biçimde ağaca yerleştirilecek. Bu sefer hiçbir yer değiştirme yapılmadı, kelimeler geliş sırasıyla eklendi:

1. “computers” köke konur.
2. “are”: a, c’den önce geldiği için sola.
3. “an”: “computers”tan küçük, sola. “are”dan da küçük, çünkü ikinci harfe bakılır ve n, r’den önce gelir. “are”nın soluna.
4. “important”: “computers”tan büyük, sağa.
5. “technological”: “computers”tan ve “important”tan büyük (t, i’den sonra gelir). “important”ın sağına.
6. “tool”: c’den ve i’den büyük. “technological” da t ile başlar ama ikinci harfler e ve o’dur; o daha sonra gelir. “technological”ın sağına.

Arama: “tennis” ağaçta var mı?

- “computers”tan büyük, sağa.
- “important”tan büyük, sağa.
- “technological” ile ilk iki harf aynı. Üçüncü harfler c ve n; n daha sonra gelir. Büyük, sağa.
- “tool” ile karşılaştır: “tennis” daha küçük, sola gidilmeli. “tool”un solunda hiçbir kelime yok, “tool” bir yapraktır.

Sonuç: “tennis” bu ağaçta yoktur. Eklenmek istenseydi yeri “tool”un sol çocuğu olurdu.

Hoca bu örneğin amacını şöyle açıkladı: Elinizde küçük de olsa bir sözlük ya da kelimelerden oluşan bir doküman olabilir. Bunu, bir kelimeyi kolayca arayabileceğiniz biçimde bir ağaca yerleştirmeniz istenebilir. En güzeli kelimeleri alfabetik sıraya göre yerleştirmektir. İkili ağaç bunun en güzel durumudur: Her adımda sağa mı sola mı gidileceğine bakılarak ilerlenir.

5.2 Dengeli ağaç

Yüksekliği h olan bir ağaçta bütün yaprakların seviyesi h ya da $h-1$ ise ağaç dengelidir.

- Birinci örnekte $h = 3$. Yaprakların seviyeleri 3, 3, 3, 2, 2. Köke en yakın yaprakla fark 1. Ağaç dengelidir.
- İkinci örnekte $h = 4$. Yaprakların seviyeleri 4, 3, 3 ve 2. En içteki yaprağın seviyesi 2, fark 2. Ağaç dengesizdir.

5.3 İkili aramanın mantığı: sayı tutma oyunu

Hoca, dengeli ikili arama ağacı kurmayı anlatmadan önce aramanın mantığını bir ağaç üzerinde gösterdi. Kök 50’ydi. Sol tarafta 25, 15, 10, 35, 30; sağ tarafta 75, 65, 95, 98 vardı.

- Tutulan sayı 93: 50’den büyük mü? Evet, ağacın sol tarafı artık yok sayılır. 75’ten büyük mü? Evet. 95’ten büyük mü? Hayır, sola gidilir. Orada düğüm yok. 93 ağaçta yoktur.
- Tutulan sayı 30: 50’den büyük mü? Hayır, sola. 25’ten büyük mü? Evet, sağa. 35’ten büyük mü? Hayır, sola. 30 mu? Evet, bulundu.

Sıralı bir dizide ikili arama da aynı mantıkla çalışır. Dizi 1, 3, 4, 5, 6, 10, 16, 17, 19 (9 eleman) için ortadaki eleman 6’dır. “6’dan büyük mü?” sorusuna evet denirse 6 ile 19 arası ikiye bölünür, 16’ya gelinir. “16’dan büyük mü?” sorusuna evet denirse 16 ile 19 arası bölünür, 17 sorulur. Amaç, aynı yarıya bölme hareketini ağaç üzerinde de yapabilmektir.

6. Sıralı Diziden Dengeli İkili Arama Ağacı Kurmak

Hoca, bu konuyu bu sınıfa daha önce anlatmadığını fark ettiğini söyleyip özür diledi ve konuyu burada anlattı.

Dizi: 1, 3, 4, 5, 6, 10, 16, 17, 19. Elemanlar sırayla eklenir. Her eklemeden sonra iki kural kontrol edilir:

- **Kural 1 (önce sol):** Bir düğümün sol çocuğu olmadan sağ çocuğu olmaz. Ağaç sol, düğüm, sağ (left, node, right) diye okunduğu için önce sol dal doldurulur.
- **Kural 2 (terazi):** Ana kökün solunda ve sağında aynı sayıda eleman kalmalıdır. Eleman sayısı tekse en fazla bir fark olabilir. Hoca bunu teraziye benzetti: "Ben bir arama yaparken bu tarafa gidersem bingo, bir kerede buluyorum; öbür taraftan devam edersem daha fazla sorgu yapıyorum. Eşit sorgu yapayım, adil olsun."

Kural bozulursa elemanlar sıralamayı koruyarak bir konum "yukarı push edilir": sağdaki eleman köke çıkar, kök sola iner, diğerleri sırayla kayar.

> **Hocanın vurgusu:** Bir alt ağaçta sol dal dolmadan sağ dala eleman konmaz. Bundan sonra hep buna dikkat edin.

Adımlar (ara ağaçlar hocanın sözlü anlatımından kuruldu):

1. **1, 3, 4:** En klasik ikili ağaç. Kök 3, sol 1, sağ 4.
2. **5 eklenir:** $5 > 3$, $5 > 4$, 4'ün sağına konur. Yükseklik farkı 1, ağaç dengeli görünür. Kökün solunda 1, sağında 2 eleman var; 1,5 ve 1,5 yapılamayacağı için bu kabul edilir. Ama 4'ün sol çocuğu yokken sağ çocuğu var, Kural 1 bozuldu. Alt ağaçta sağdaki eleman yukarı çıkar, 4 onun soluna iner. Yeni durum: kök 3, sol 1, sağ 5; 5'in sol çocuğu 4.
3. **6 eklenir:** $6 > 3$, $6 > 5$, 5'in sağına. Kökün solunda 1, sağında 3 eleman var. Toplam 4 eleman 2'ye 2 bölünebilir. Herkes bir yukarı push edilir. Yeni durum: kök 4; sol alt ağaç 3 ve onun sol çocuğu 1; sağ alt ağaç 6 ve onun sol çocuğu 5. Hoca 3'ü önce yanlış dala indirdiğini, sol olmadan sağ olamayacağı için düzelttiğini belirtti. Kökün iki yanında 2'şer eleman var.
4. **10 eklenir:** $10 > 4$, $10 > 6$, 6'nın sağına. Solda 2, sağda 3 eleman. 2,5 ve 2,5 yapılamaz, sol dallar da dolu. Değişiklik yok.
5. **16 eklenir:** $16 > 4$, $16 > 6$, $16 > 10$, 10'un sağına. 10'un sol çocuğu yok, Kural 1 bozuldu: 16 yukarı çıkar, 10 onun soluna iner. Kökün solunda 2, sağında 4 eleman var; 6 eleman 3'e 3 bölünmeli. Yine herkes bir yukarı push edilir: 10 alt ağaç köküne geçer, 6 onun soluna iner, 5 köke çıkar, 4 aşağı iner. Yeni durum: kök 5; sol alt ağaç 3 (sol 1, sağ 4); sağ alt ağaç 10 (sol 6, sağ 16). İki tarafta 3'er eleman.
6. **17 eklenir:** $17 > 5$, $17 > 10$, $17 > 16$, 16'nın sağına. 16'nın solu boş, Kural 1 bozuldu: 17 yukarı çıkar, 16 onun soluna iner. Solda 3, sağda 4 eleman. Fark 1, değişiklik yapılmaz.
7. **19 eklenir:** $19 > 5$, $19 > 10$, $19 > 17$, 17'nin sağına. Solda 3, sağda 5 eleman; toplam 8, 4'e 4 olmalı. Herkes bir yukarı push edilir: 19 alt ağaç köküne, 17 onun soluna, 16 10'un yerine, 10 6'nın yerine, 6 5'in yerine (köke), 5 4'ün yerine, 4 3'ün yerine, 3 aşağı iner, 1 de 3'ün sol çocuğu olarak kalır. Hoca bu kaydırmaya "astral seyahat" dedi.

Son ağaç:

Düğüm	Sol çocuk	Sağ çocuk
6 (kök)	4	16
4	3	5
3	1	-
16	10	19
19	17	-

Kökün iki yanında 4'er eleman vardır. Bu ağacın içten sıralı (inorder) okunuşu 1, 3, 4, 5, 6, 10, 16, 17, 19'dur, yani sıralama korunmuştur.

Arama örnekleri:

- 17 aranır: 6'dan büyük, 16'dan büyük, 19'dan küçük, 17'ye gelinir. Bulundu.
- 2 aranır: 6'dan küçük, 4'ten küçük, 3'ten küçük, 1'e gelinir. 2 ağaçta yoktur.

Hocaya göre dizi üzerinde aramayla ağaç üzerinde arama arasında hiçbir fark yoktur. Sıralı dizide de ilk soru 6, sonra 16 ya da 4 olurdu.

> **Ders sonrası not:** Derste anlatılan "her eklemeden sonra elemanları yukarı itme" yöntemi standart ikili arama ağacı eklemesi değildir. Standart eklemede yeni eleman yaprağa eklenir ve ağaç dengesiz kalabilir. Sıralı bir diziden dengeli ağaç kurmanın büyük olasılıkla daha kısa yolu, ortadaki elemanı kök yapıp sol ve sağ yarılara aynı işlemi uygulamaktır. Bu dizide ortadaki eleman 6'dır ve hocanın son ağacının kökü de 6'dır.

7. İkili Ağacın Dizide Tutulması

Soru (hocanın kendi sorusu): Ağaçta yukarı itmek, sağa sola çekmek görsel olarak kolay. Peki bunu nasıl kodlayacağım?

Cevap: Ağacın her gözüne mantıksal bir adres (indis) verilir ve ağaç tek boyutlu bir diziye indirgenir.

Hoca daha küçük bir dizi kullandı: 23, 13, 19, 47, 11, 43, 7, 29, 52. Bu dizi ikili arama ağacına uygun sırayla verilmişti, böylece doğrudan yerleştirilebildi. 23'ten küçükler ağacın sol tarafında, büyükler sağ tarafında; 13'ten küçükler 13'ün solunda, büyükler sağında.

İndis kuralı:

- Kökün adresi 1 olsun. Bazı kaynaklar 0'dan başlatır. Aynı kural uygulandıktan sonra bunun önemi yoktur.
- i indisli düğümün sol çocuğu $2i$ indisindedir.
- Sağ çocuğu $2i + 1$ indisindedir.

Hoca, ikili ağaçta seviyelerin $2^0, 2^1, 2^2, 2^3$ eleman alabildiğini hatırlattı. Ağaç tam dolu olsaydı adresler soldan sağa 1'den 15'e kadar giderdi. Ağaç dolu olmadığı için yalnız dolu gözler kullanılır, ama boş gözlerin adresleri de kurala göre ayrılmıştır (ör. bir düğüm gelseydi 11, 12, 13, 14, 15 numaralı gözler).

İndis	1	2	3	4	5	6	7	8	12
Değer	23	13	47	11	19	43	52	7	29

43'ü aramak:

1. İndis 1'de 23 var. 43, 23'e eşit değil ve ondan büyük. Sağa gidilir: $2 \cdot 1 + 1 = 3$.
2. İndis 3'te 47 var. $43 < 47$. Sola gidilir: $2 \cdot 3 = 6$.
3. İndis 6'da 43 var. Bulundu.

Bulunmasaydı ve aranan sayı daha küçük olsaydı $2 \cdot 6 = 12$, daha büyük olsaydı $2 \cdot 6 + 1 = 13$ numaralı göze bakılacaktı.

Silmek ve eklemek: Hoca, veri yapıları dersinde bunlarla ilgili ödevler verileceğini söyledi: ikili ağaca eleman ekle, ağaçtan eleman çıkar.

- 52 çıkarılmak istenirse 52'nin bulunduğu göz sıfırlanır (52 bir yapraktır).
- 50 eklenmek istenirse: $50 > 23$, $50 > 47$, $50 < 52$. 52 indis 7'dedir. 50, $2 \cdot 7 = 14$ numaralı göze yazılır.

8. Ağaçta Gezinme (Traversal)

Aradan sonra hoca yeni bir konuya geçti. Ağaçlarda tutulan veriler belirli kurallarla okunur. Ağaç üzerindeki bu gezinmeye traversal denir. Hocaya göre Türkçede tek kelimelik bir karşılığı yoktur. Kitaplarda dört yol vardır; üçü önemlidir.

Aşağıda $T_1, T_2, \ldots, T_n$ kökün soldan sağa alt ağaçlarıdır.

- **Önden sıralı (preorder):** Önce kök, sonra $T_1, T_2, \ldots$ soldan sağa. Her alt ağaç da kendi içinde preorder okunur: önce o alt ağacın kökü, sonra onun alt ağaçları.

- **İçten sıralı (inorder):** Önce T_1, sonra kök, sonra $T_2, T_3, \ldots$. Her alt ağaçta da önce sol dal, sonra kök gelir.
- **Sondan sıralı (postorder):** Önce $T_1, T_2, \ldots$ (çocuklar), en sonda kök. Alt ağaçlarda da önce çocuklar, sonra kök.

Slayttaki dördüncü yol postorder'ın tersiydi (reverse). Hoca "yok kabul edin, ilgilenmiyoruz" dedi.

8.1 Örnek ağaç

Hoca, çizmenin "gerçekten çok zor" olduğunu söylediği bir ağaç verdi ve öğrencilerden kağıt kalemle okumalarını istedi:

- a'nın çocukları: b, c, d.
- b'nin çocukları: e, f.
- e'nin çocukları: j, k.
- k'nin çocukları: n, o, p.
- d'nin çocukları: g, h, i.
- g'nin çocukları: l, m.

8.2 Preorder okuma

Önce kök a okunur. Sonra a'nın ilk çocuğu b'ye geçilir. b "sanki bambaşka bir ağaçmış gibi" düşünülür: önce kökü b okunur, sonra çocukları. b'nin ilk çocuğu e'dir; e okunur, sonra e'nin çocukları j ve k. k bir alt ağacın köküdür: önce k, sonra n, o, p. b'nin ikinci çocuğu f okunur. a'nın ilk çocuğu bitti; ikinci çocuğu c okunur. Üçüncü çocuk d bir alt ağaçtır: d, sonra g ve g'nin çocukları l, m, sonra h ve i.

Sonuç: a, b, e, j, k, n, o, p, f, c, d, g, l, m, h, i.

Hoca: "Bu kalıbı gördüğüm her alt ağaçta aynı şeyi uygulayarak gidiyorum."

8.3 Inorder okuma

Önce alınan ağacın sol dalına inilir. Sol dalda bir ağaç varsa onun soluna, onun da soluna gidilir. Sonra köke çıkılır, sonra diğer çocuklar okunur.

a'nın sol dalı b'nin alt ağacıdır, onun da sol dalı e'nin alt ağacıdır. e'nin solu j okunur, sonra kök e. Sonra e'nin sağ çocuğu k'nin alt ağacı inorder okunur: n, k, o, p. b'nin sol tarafı (T1) bitti; kök b, sonra f. a'nın sol kolu bitti; kök a okunur. Sonra c; c'nin altı boş olduğu için sonraki dala geçilir. d'nin alt ağacında önce sol çocuk g'ye gidilir; g'nin de solu olduğu için önce l, sonra g, sonra m. Sonra d, sonra h ve i.

Sonuç: j, e, n, k, o, p, b, f, a, c, l, g, m, d, h, i.

"Hep alt ağaca inip orada verilmiş okuma kuralı neyse onu okuyoruz. Bittikçe yukarı çıkıyoruz."

8.4 Postorder okuma

Önce hep yapraklar (çocuklar) okunur, en son kök. b'ye, sonra e'ye inilir. e'nin ilk çocuğu j okunur. Sonra k'nin çocukları n, o, p ve onların kökü k. Sonra e, sonra f, sonra b. Sonra c. Sonra d'nin çocukları: önce g'nin çocukları l, m, sonra g, sonra h, i, sonra d. En son kök a.

Sonuç: j, n, o, p, k, e, f, b, c, l, m, g, h, i, d, a.

> **Ders sonrası not:** Postorder okumayı sözlü yapan öğrencinin cevabı altyazıda "M, G, E, İ" diye bozuk duyuluyor. Doğru sıra yukarıdaki gibidir: l, m, g, h, i, d, a. Üç sıralama da Python ile doğrulandı.

9. Huffman Kodlaması

Hoca bu konunun "gerçekten çok önemli" olduğunu söyledi. Konuyu normalde tahtada anlattığını, tahta olmadan anlatması zor olduğu için sunuma geçirdiğini belirtti. Huffman kodları

aynı zamanda Huffman ağacı (Huffman tree) olarak geçer. Bu ağacı oluşturmak için Huffman algoritması kullanılır.

9.1 Neden sıkıştırma? Renk örneği

Huffman kodları farkında olmadan kullanılır. Örneğin JPEG bir sıkıştırma formatıdır: resmi sıkıştırır, boyutunu küçültür. Resimlerde üç renk paleti (kanalı) vardır: kırmızı, yeşil, mavi (red, green, blue). Her biri 0-255 arasında bir değer alır ve bir bayt ile ifade edilir. Üçü de 0 ise zifiri siyah, üçü de 255 ise beyaz çıkar. Diğer bütün kombinasyonlar farklı renklerdir.

Hoca, insan gözünün bu kadar farklı rengi ayırt edemediğini söyledi. Örneğin (15, 75, 46), (15, 75, 47) ve (15, 75, 48) renkleri arasındaki farkı saatlerce baksanız ayırt edemezsiniz. O zaman bu üç renk tutulmak yerine hepsine ortalamaları olan (15, 75, 47) verilebilir. Birkaç rengi tek renge indirgemek bir sıkıştırma işlemidir. JPEG dosyası resmin sıkıştırılmış formatıdır. Bu sıkıştırmanın birçok adımı vardır ve bunlardan biri Huffman kodlarını kullanır. Huffman kodları oluşturulurken kullanılan ağaçlara da Huffman ağaçları denir.

> **Ders sonrası not:** Hoca her renk kanalı için "7 bit" deyip $3 \cdot 7 = 21$ bitten 2^{21} renk hesapladı. Oysa kendisinin de söylediği gibi her kanal 0-255 arası, yani 8 bittir. Toplam 24 bit ve $2^{24} = 16\,777\,216$ renk vardır ($2^{21} = 2\,097\,152$ olurdu).

9.2 Metin örneği: neden değişken uzunluklu kod?

Hoca bunun dersin içeriği olmadığını, ama genel kültür olarak bilinmesini istediğini söyledi. ASCII tablosunda her karakter bir bayttır (8 bit). Bir dokümanda a'dan 25, z'den 2 adet kullanılmış olsun.

- Her karakter 8 bit: $25 \cdot 8 = 200$ bit, $2 \cdot 8 = 16$ bit. Toplam 216 bit.
- a 2 bitle, z 10 bitle kodlansa: $25 \cdot 2 = 50$ bit, $2 \cdot 10 = 20$ bit. Toplam 70 bit.

Yarıdan daha az bit harcanır. Fikir: Çok kullanılan karakteri daha az bitle, az kullanılanı daha çok bitle kodlamak. Ödev gönderirken kullanılan RAR gibi dosya sıkıştırmalarında da bir sıkıştırma algoritması vardır ve hocaya göre bunların içinde Huffman kodları kullanılabilir.

9.3 Huffman dizisinin kurulması

Semboller ve frekansları: a(25), e(14), k(8), l(7), m(4), z(2). Toplam 60 karakter, 6 sembol.

1. **Sırala:** Frekanslar küçükten büyüğe sıralanır: 2, 4, 7, 8, 14, 25.
2. **En küçük iki değeri topla:** $2 + 4 = 6$. Toplam, sıralamayı bozmayacak şekilde yerleştirilir (4 ile 7 arasına). Toplamlar slaytta kırmızıyla yazıldı.
3. **Devam et:** $6 + 7 = 13$, $8 + 13 = 21$, $14 + 21 = 35$, $25 + 35 = 60$. Her toplam sıralamayı bozmadan yerleştirildi. Toplanacak bir şey kalmayınca durulur.

Hoca anlatırken bir an 7 ile 8'i toplayıp 15 yazdı, sonra "aklınızı karıştırmak istemiyorum" diyerek düzeltti ve doğru sırayla (6 ile 7) devam etti.

> **Hocanın vurgusu:** Öğrencilerin yaptığı hata şu: 2 ile 4'ü topluyor, sonra gidip 4'ü 6 ile topluyor. Hayır. Toplama girmiş bir değer tekrar toplamaya sokulmaz. 2 ile 4 toplandı, bitti. 6 ile 7 toplandı, artık bunlar da bitti.

Kırmızılar (toplamlar) ağacın **ara düğümlerini**, siyahlar (gerçek frekanslar) ağacın **yapraklarını** oluşturur.

Neden eksi işareti? Kod yazarken bir sayıyı kırmızı yazmak bilgisayara bir şey anlatmaz. Bilgisayara bunun bir ara toplam olduğunu anlatmak için toplamların önüne eksi konur. Kod eksi bir sayı gördüğünde onun ara toplam olduğunu bilir. Toplama yine mutlak değerce yapılır: 6 ile 7 toplanınca 13 yazılır (eksiyle toplansa $-6 + 7 = 1$ ederdi, öyle yapılmaz). Eksi yalnız kodda ara düğümü işaretlemek içindir.

Son dizi: 2, 4, -6, 7, 8, -13, 14, -21, 25, -35, -60.

Duruş noktası: n sembol varsa dizi $2n-1$ elemana genişletilir. Burada $2 \cdot 6 - 1 = 11$ eleman. Bir for ya da while döngüsünde duruş koşulu budur.

9.4 Huffman ağacının kurulması

Ağaç dizinin **sonundan** kurulur.

1. Son eleman -60 köke konur. Eksi olduğu için ara düğümdür, ondan dallar çıkacaktır. -60, 35 ile 25'in toplamıdır. İlk okunan -35 sola, ikinci okunan 25 sağa konur. 25 siyah, yani gerçek değerdir; yaprak olarak kalır.
2. -35, 21 ile 14'ün toplamıdır. İlk okunan -21 sola, 14 sağa. 14 yapraktır.
3. -21, 13 ile 8'in toplamıdır. -13 sola, 8 sağa. 8 yapraktır.
4. -13, 7 ile 6'nın toplamıdır. İlk okunan 7 sola, -6 sağa. 7 yapraktır.
5. -6, 4 ile 2'nin toplamıdır. 4 sola, 2 sağa. İkisi de yapraktır.

Bütün dizi ağaca yerleştirildi. Bu ağaç Huffman ağacıdır.

Hoca şu hatayı da gösterdi: Dikkatsizce -60'ın altına -60 ve -35 yazılırsa ağaç hatalı olur, çünkü dizide toplamı 60 veren başka bir ikili bulunamaz.

> **Hocanın vurgusu:** İlk okuduğum sola, ikincisi sağa. Bunu sürekli söylüyorum, çünkü bazısında ilk okuduğunuzu sola, bazısında sağa koyarsanız kod yazamazsınız. Her şey karışır. Bir kural olmalı ve o kurala göre hareket etmelisiniz.

9.5 Kodların okunması

Ağaçtaki her **sol** dala 0, her **sağ** dala 1 yazılır. Kod, kökten yaprağa doğru (yukarıdan aşağı) okunur.

Sembol	Frekans	Kod	Bit	Toplam bit
a	25	1	1	25
e	14	01	2	28
k	8	001	3	24
l	7	0000	4	28
m	4	00010	5	20
z	2	00011	5	10

- a: kökten a'ya bir kenar geçilir, değeri 1.
- e: 0 ve 1.
- k: 0, 0, 1.
- l: dört tane 0.
- m: üç tane 0, sonra 1 ve 0.
- z: üç tane 0, sonra 1 ve 1.

Kazanç:

- Toplam karakter: $25 + 14 + 8 + 7 + 4 + 2 = 60$.
- 8 bitlik kodla: $60 \cdot 8 = 480$ bit.
- Huffman koduyla: $25 + 28 + 24 + 28 + 20 + 10 = 135$ bit.

Hoca: "480 bit mi harcamak istersiniz, 135 bit mi? Tabii ki 135." Ağacın adı Huffman ağacıdır, çünkü rastgele değil, belirli bir kuralla oluşturulur.

9.6 Neden yukarıdan aşağı okunur?

Soru (hocanın kendi sorusu): Hocam niye yukarıdan aşağı okuduk da aşağıdan yukarı okumadık?

Cevap: Aşağıdan yukarı okunsaydı kodlar şöyle olurdu: a = 1, e = 10, k = 100, l = 0000, m = 01000, z = 11000. Sıkıştırılmış bir dosyanın geri açılabilmesi gerekir. RAR açıldığında gönderilenin dışında bir şey çıkmamalıdır.

Hoca "ders için önemli değil ama bilmeniz güzel bir şey olur" diyerek bir örnek verdi. Sıkıştırmak, kodları arka arkaya eklemektir. Her 8 biti bir bayt değeri olarak yazınca absürt bir şey çıkar; sıkıştırılmış dosya açılmadan okunamaz.

- **Doğru kodlarla (mavi):** “k a” dizisi 001 ve 1, yani `0011` olur. Açarken: 0 diye bir kod var mı? Yok. 00 diye bir kod var mı? Yok. 001 var mı? Evet, k. Sonra 1 geliyor; 1 diye bir kod var mı? Evet, a. Dizi çözüldü.
- **Ters kodlarla (kırmızı):** “k a” dizisi 100 ve 1, yani `1001` olur. Açarken: 1 diye kod var mı? Evet, a. Sonra 0 var mı? Yok. 00 var mı? Yok. 001 var mı? Yok. Geri kalanı çözülemedi.

Sebep: Bir kodun değeriyle başlayan başka bir kod olmamalıdır. Ters okumada 1 hem a’nın kodu hem de e’nin (10), k’nin (100) ve z’nin (11000) başlangıcıdır. Açıcı başta 1’i görünce a sanar. 10 ile başlayan kod da e ile k’yi karıştırır. Bu yüzden kod yukarıdan aşağıya oluşturulmalıdır.

> **Ek bilgi (derste anlatılmadı):** Hiçbir kodun başka bir kodun başlangıcı olmadığı kodlara önek kodu (prefix code) denir. Huffman kodu bir önek kodudur, bu yüzden bit dizisi soldan okunarak tek biçimde çözülür. Ayrıca eşit toplamlar olduğunda farklı Huffman ağaçları ve farklı kodlar çıkabilir, ama toplam bit sayısı yine en küçük değerdir.
>
> **Ders sonrası not:** Kod uzunlukları, 135 ve 480 bit, 216 ve 70 bit, doğru kodların önek kodu olduğu, ters kodların olmadığı ve `0011` dizisinin “k a” diye çözüldüğü Python ile doğrulandı. Standart bir öncelik kuyruğu (heap) ile kurulan Huffman ağacı da 135 bit verdi.

10. Kapsayan Ağaçlar (Spanning Tree)

İkinci aradan sonra hoca kapsayan ağaçlar (spanning tree) konusuna geçti. “Span” kelimesinin anlamını öğrencilere sordurdu; bir öğrenci “mesafe” gibi anlamlar çıktığını söyledi. Hoca, iki düğüm arasındaki yol için farklı kaynaklarda farklı terimler (edge, span, path gibi) kullanıldığını, bunların Türkçede tek bir şeye, kenara karşılık geldiğini belirtti.

Kapsayan ağaç: Kapalı döngüleri olan bir G grafı verilmiş olsun. Bu grafın içinden çıkarılabilen ve

- G’nin bütün düğümlerini içeren,
- döngü oluşturmayan,
- G’nin kenarlarından oluşan

her alt ağaç bir kapsayan ağaçtır. Slaytta 6 düğümlü bir graftan kalın çizilmiş bir kapsayan ağaç çıkarıldı; 6 düğümün hepsi oradaydı. Hoca ayrıca 4 düğümlü bir graf çizip dört düğümün dördünü de içeren, ama farklı şekillerde bağlanmış birçok kapsayan ağaç gösterdi.

En küçük kapsayan ağaç fikri: Düğümler bir köydeki evler olsun. Evler su ya da kanalizasyonla birbirine bağlanacak. Kenarlardaki sayılar evler arasındaki mesafelerdir; o kadar boru ya da elektrik teli döşenecektir. Bu bağlantılar farklı şekillerde yapılabilir, ama amaç en az maliyettir. Toplam mesafesi en küçük bağlantı bulunursa karşımıza en küçük kapsayan ağaç (minimum spanning tree) çıkar.

Bir başka problem: Kapsayan ağaçları gözle görerek çizebiliriz, ama bunu bilgisayara yaptırmak için algoritma gerekir. Kapsayan ağaç oluşturmada en popüler yöntemler geniş öncelikli arama (breadth first search) ve derin öncelikli arama (depth first search) yöntemleridir.

11. Derin Öncelikli Arama (DFS)

Derin öncelikli arama (Depth First Search, DFS) şöyle çalışır:

1. Bir düğüm kök kabul edilir ve ondan başlanır.
2. Gidilebilecek en uzak noktaya kadar gidilir.
3. Gidecek yer kalmayınca geri dönülür.
4. Gezilmemiş bir düğüme geçilir ve ondan gidilebilecek en uzak noktaya kadar gidilir.
5. Bütün düğümler gezilene kadar devam edilir.

Bu algoritmayı yazmak için yığın (stack) kullanılır. Yığında iki komut vardır: push (itmek, yığına bir şey atmak) ve pop (çekmek, yığından almak). Başlangıç düğümüne kendiniz karar verebilirsiniz. Hoca düğümleri alfabetik sıraladığı için A’dan başladı.

Slayttaki graf: A’nın komşuları D, H, B; D’nin komşusu F; B’nin komşuları E ve C.

Tabloda sırasıyla işlem, yığının o anki durumu, yığından çıkan değer ve çıktı (gezilen düğümler) yazıldı:

Adım	İşlem	Yığın	Çıkan	Çıktı
1	Push(A)	A	-	A
2	Push(D)	D-A	-	D
3	Push(F)	F-D-A	-	F
4	Pop()	D-A	F	-
5	Pop()	A	D	-
6	Push(H)	H-A	-	H
7	Pop()	A	H	-
8	Push(B)	B-A	-	B
9	Push(E)	E-B-A	-	E
10	Pop()	B-A	E	-
11	Push(C)	C-B-A	-	C
12	Pop()	B-A	C	-
13	Pop()	A	B	-
14	Pop()	-	A	-

Adım adım anlatım:

- A'ya gelindi, kırmızıya boyandı: artık bu düğüme bir daha gelinmeyecek. A yığına push edildi ve çıktıya yazıldı.
- A'nın üç komşusundan D seçildi. D yığına atıldı (A'nın üstüne). D, yığından ilk çıkacak eleman oldu. Çıktıya D yazıldı.
- D'den F'ye geçildi. F yığına atıldı, çıktıya yazıldı.
- F'den gidilecek yer yok: pop, F çıktı. D'ye dönüldü; D'den de gidilecek yer yok: pop, D çıktı. Yığında yalnız A kaldı.
- A'nın kalan komşuları H ve B. H seçildi, push edildi, çıktıya yazıldı. H'den gidilecek yer yok: pop.
- A'dan kalan komşu B. B push edildi, çıktıya yazıldı.
- B'nin komşuları E ve C. E seçildi, push, çıktı. E'den gidilecek yer yok: pop. Yığında B-A kaldı.
- B'den kalan komşu C. C push, çıktı. C'den gidilecek yer yok: pop. B'den de gidilecek yer kalmadı: pop. A'dan da: pop. Yığın boşaldı.

Çıktı: A, D, F, H, B, E, C.

Hoca özetinde şöyle anlattı: A'dan gidilebilen en uzak noktaya kadar gidildi (D, F). Geri dönüldü. H seçildi, başka yer kalmadı, A'ya dönüldü. B'ye gidildi, E seçildi, B'ye dönüldü, C'ye gidildi, B'ye ve A'ya dönüldü.

> **Ders sonrası not:** Hoca slaytı özetlerken çıktıyı "A, D, F, H, B, C" diye okudu ve E'yi atladı. Tablodaki adımlara göre doğru çıktı A, D, F, H, B, E, C'dir. Python ile doğrulandı.

12. Geniş Öncelikli Arama (BFS)

Geniş öncelikli arama (Breadth First Search, BFS) düğümleri seviye seviye ziyaret eder. Hocanın benzetmesi: Suya bir taş attığınızda önce küçük bir halka oluşur, sonra daha büyükleri; halkalar büyüye büyüye kaybolur. Algoritma da önce başlangıç düğümünün bir sonraki seviyedeki komşularına bakar. O seviye bitmeden bir sonraki seviyeye geçmez.

BFS'de kuyruk (queue) kullanılır. Kuyrukta ilk giren ilk çıkar.

Slayttaki graf: A'nın komşuları B ve C; B'nin komşusu F; C'nin komşuları E ve D; D'nin komşusu H; H'nin komşusu J. Başlangıç A.

Adım	Eklenen	Çıkan	Kuyruk	Çıktı
1	A	-	A	A

Adım	Eklenen	Çıkan	Kuyruk	Çıktı
2	B-C	A	B-C	B-C
3	F	B	C-F	F
4	E-D	C	F-E-D	E-D
5	-	F	E-D	-
6	-	E	D	-
7	H	D	H	H
8	J	H	J	J
9	-	J	-	-

Adım adım anlatım:

- A eklendi, çıktıya A yazıldı.
- A'nın komşuları B ve C kuyruğa atıldı. A'nın seviyesi bitti, A kuyruktan çıktı. Çıktıya B ve C.
- Kuyruğa ilk giren B'ydi. B'nin komşusu F eklendi, B çıktı. Kuyrukta C ve F. Çıktıya F.
- C'nin komşuları E ve D eklendi, C çıktı. Kuyrukta F, E, D. Çıktıya E ve D.
- F'den gidilecek başka düğüm yok, F çıktı. E'den de yok, E çıktı. Kuyrukta yalnız D.
- D'nin komşusu H eklendi, D çıktı. Çıktıya H.
- H'nin komşusu J eklendi, H çıktı. Çıktıya J.
- J'den gidilecek yer yok, J çıktı.

Çıktı: A, B, C, F, E, D, H, J.

Bir seviyeden diğerine geçerken o seviyedeki düğümlerin kuyruktan çıkması gerekir. A çıkmalı ki B ve C'nin seviyesine gelinsin; B ve C çıkmalı ki bir sonraki seviyeye geçilsin.

Soru: B ve C yerine önce C sonra B'yi koysaydık çıktı değişirdi, değil mi? Bu adımda serbest miyiz?

Cevap: Evet. Bunun tek bir çözümü, tek bir çıktısı yok. Ben A'dan sonra önce B'yi sonra C'yi seçtim, ama siz algoritmanızda önce C'yi sonra B'yi seçebilirsiniz. O zaman çıktı elbette değişir. Kapsayan ağaçlarda da gösterdiğim gibi bunlar çok farklı şekillerde oluşturulabilir.

> **Ek bilgi (derste anlatılmadı):** DFS ve BFS'de bir düğümden geçilen her kenar, yani bir düğümü ilk kez keşfettiren kenar, grafın bir kapsayan ağacını oluşturur. Bu yüzden bu iki arama, ağırlıksız bir graftan kapsayan ağaç üretmenin yollarıdır. Komşuların seçilme sırası değişince farklı kapsayan ağaçlar çıkar.

13. En Küçük Kapsayan Ağaç ve Prim Algoritması

Hoca, ağaçlar konusunun bu derste bitmeyeceğini, ama en küçük kapsayan ağacı anlatacağını söyledi.

> **Hocanın vurgusu:** Bunu lütfen iyi dinleyin. Burada bir nokta var, nedense orada hep hata yapılıyor. O hatayı vurgulayacağım.

13.1 Problem

Bir graftan onlarca farklı kapsayan ağaç çıkarılabilir. Kenarların üzerinde sayısal değerler vardır. Öyle bir kapsayan ağaç bulunmalı ki kenar değerlerinin toplamı, olabilecek bütün kapsayan ağaçlar içinde en küçük olsun. Slayttaki ilk grafta hoca örnek olarak 1,8 + 1,9 + 1,9 + 2,1 + 1,8 gibi bir toplamdan söz etti. Bütün düğümlerden geçmek şartıyla çok farklı kapsayan ağaçlar çıkar; "minimum" kelimesi kenar toplamının en küçük olmasından gelir. Bu problemi çözen algoritmalardan biri Prim algoritmasıdır.

13.2 Köy ve su deposu örneği

Bir köy düşünün. 1 numaralı düğüm su deposu, diğerleri evlerdir. Depodan evlere su bağlanacak. Kenar ağırlıkları metre cinsinden uzaklıktır. Hoca çizimin orantılı olmadığını, "300 neden

700’den büyük duruyor” diye düşünülmemesini söyledi. Metre ne kadar azsa o kadar az boru ve işçilik kullanılır. Yani en küçük kapsayan ağaç bulunmalıdır.

Komşuluk matrisi: 7 düğüm olduğu için 7×7 bir matris kurulur.

- Köşegen 0’dır. Evin kendi içindeki tesisattan bahsedilmiyor.
- Bağlantı olmayan çiftlere (ör. 2 ile 3) çok büyük bir sayı yazılır ve M ile gösterilir. Algoritma hep küçük sayıyı seçtiği için M hiç seçilmez. Hoca “random büyük bir sayı, mesela 10 milyon koyun” dedi.
- Matris köşegene göre simetriktir. 1 ile 4 arası 300 ise 4 ile 1 arası da 300’dür; 1 ile 2 arası 1100 ise 2 ile 1 arası da 1100’dür.

	1	2	3	4	5	6	7
1	0	1100	100	300	600	1000	400
2	1100	0	M	M	M	M	900
3	100	M	0	M	500	M	M
4	300	M	M	0	M	700	M
5	600	M	500	M	0	200	M
6	1000	M	M	700	200	0	800
7	400	900	M	M	M	800	0

Başlangıç noktası: Prim algoritması için bir başlangıç noktası gerekir. Bunu problem belirler. Su deposu buradaysa köyün en uzağındaki evden işe başlanmaz; depodan çıkılır. Ya da soruda “1’den başlayacağız” denir.

13.3 Algoritmanın adımları

Kural: Ziyaret edilen düğümün **sütunu** karalanır (üzeri çizilir). Bu, o düğüme bir daha gelinmeyeceği anlamına gelir. Sonra ziyaret edilmiş düğümlerin **satırlarında**, üzeri çizilmemiş sütunlardaki en küçük değer aranır.

1. **Başlangıç 1.** 1. sütun çizilir. 1. satırdaki en küçük değer 100’dür (3 numaralı düğüm). Kenar 1-3 (100) seçilir. 3. sütun da çizilir. Artık 1 ve 3’ten “kazılara” devam edilir.
2. **Ziyaret edilen {1, 3}.** 1. satırda sıradaki en küçük 300, 3. satırda en küçük 500. Daha küçüğü 300: kenar 1-4 seçilir. 4. sütun çizilir.
3. **Ziyaret edilen {1, 3, 4}.** 4. satırda en küçük 700, 3. satırda 500 (700’den küçük, akılda tutulur). 1. satırda üzeri çizilmemiş sütunlarda 400 var, 500’den de küçük. Kenar 1-7 (400) seçilir. 7. sütun çizilir.
4. **Ziyaret edilen {1, 3, 4, 7}.** 1. satırda en küçük 600. 3. satırda 500 var, daha küçük. 4. ve 7. satırlarda 500’den küçük yok. Kenar 3-5 (500) seçilir. 5. sütun çizilir.
5. **Ziyaret edilen {1, 3, 4, 5, 7}.** 1. satırda en küçük 1000. 3’te 1000’den küçük yok. 4’te 700 var. 5’te 200 var. 7’de 200’den küçük yok. Kenar 5-6 (200) seçilir. 6. sütun çizilir.
6. **Ziyaret edilen {1, 3, 4, 5, 6, 7}.** Kalan tek sütun 2. 1. satırda 1100; 3, 4, 5, 6. satırlarda M; 7. satırda 900. Kenar 7-2 (900) seçilir. Bütün düğümler gezildi.

Hoca 5. adımda 200’ü sütun çizilmeden önce işaretlediğini fark edip “bu sonradan gelecekti, şu 200’ün olmadığını düşünün” diye düzeltti.

Kapsayan ağacın çizilmesi: Seçilen kenarlar grafta kalın çizilir: 1-3 (100), 1-4 (300), 1-7 (400), 3-5 (500), 5-6 (200), 7-2 (900).

Toplam: $100 + 300 + 400 + 500 + 200 + 900 = 2400$.

Birimi belirtilmediği için hoca “2400 birim diyelim” dedi. Bütün düğümlerden geçen ve toplamı 2400’den küçük bir kapsayan ağaç bulunamaz. Hocaya göre Prim algoritması bunu en güzel şekilde veren algoritmadır.

> **Hocanın vurgusu:** Şunu unutmayın: Üzerinden geçtiğim düğümlerin sütununa bir daha küçük sayı var mı diye bakmıyorum. Yalnız gezdiğim düğümlerin satırlarında, üzeri çizilmemiş olanlarda en küçük sayıyı arıyorum.
>
> **Ders sonrası not:** Prim sonucu (kenarlar ve 2400 toplamı) Python ile doğrulandı. Aynı matrise Kruskal algoritması uygulandığında da toplam 2400 çıktı.

13.4 Görsel demo: Japonya haritası

Hoca anlamayanlar için internette hazır bir sayfa gösterdi (sayfanın hâlâ yayında olup olmadığını bilmediğini söyledi). Haritada Japonya'daki şehirler ve aralarındaki yollar vardı; hoca bunun metro hattı olabileceğini söyledi. Prim algoritması adım adım görselleştiriliyordu:

- Tokushima'dan başlandı. Seçenekler 140 ve 85; küçük olan 85 seçildi.
- Ulaşılan düğümlerden çıkan yolların (kırmızılar) en küçüğü seçilerek devam edildi: 19 ile 39 arasından küçük olan, sonra 12 ile 126 arasından 12 gibi.
- Bir düğüme daha kısa bir yeni yol açılırsa eski uzun yol iptal edilir. Örneğin Kyoto'ya 105'lik yol yerine 74'lük yol aktif oldu; Tottori'ye 70 yerine 31'lik yol geldi; Wadayama'dan 30'luk yol açılınca Kyoto'dan gelen 74 iptal edildi.
- Eşit değerler (iki tane 65) olduğunda bunlardan biri tercih edilir.

Hoca demoda birkaç kez yanlış oka basıp geri gittiğini söyledi. Bu demonun, matrisle yapılan işlemin görsel hali olduğunu, istenirse matrise geçirilerek aynı sonucun elde edilebileceğini belirtti.

> **Ek bilgi (derste anlatılmadı):** Demoda görülen "daha kısa yol açılınca eski yolu iptal etme" adımı, Prim algoritmasının yaygın uygulamasıdır: Her ziyaret edilmemiş düğüm için ağaca bağlanan en küçük kenar tutulur ve yeni bir düğüm eklendiğinde bu değerler güncellenir. Matris yöntemindeki "ziyaret edilmiş satırlarda en küçük değeri arama" ile aynı sonucu verir.

Örnek Sorular

Soru 1 (derste sözlü, 00:27:44). Slayttaki ikili ağaçta kök A verilmiştir. A'nın kaç çocuğu var, hangileri? D'nin sağ çocuğu var mı? Ara düğümler ve uç düğümler hangileri? Ağacın yüksekliği nedir?

> **Çözüm (derste verildi):** A'nın iki çocuğu var: sol B, sağ C. D'nin sağ çocuğu yok. Ara düğümler A, C, D, E, H; uç düğümler B, F, G, I. Yükseklik 4.

Soru 2 (derste sözlü, 00:36:51). Tam ikili ağaçta $t = 5$ uç düğüm var. $\log_2 5$ nedir ve yükseklik koşulu sağlanıyor mu?

> **Çözüm (derste verildi):** $\log_2 5 \approx 2{,}32$ ("iki küsür"). Ağacın yüksekliği 3'tür ve $2{,}32 \leq 3$. Ağaç 8 uç düğüme tamamlanırsa $\log_2 8 = 3 = h$.

Soru 3 (derste sözlü, 00:40:58). i ara düğümlü tam ikili ağaçta toplam düğüm sayısı $2i + 1$ ise, slayttaki ağaçta kaç düğüm olmalıdır?

> **Çözüm (derste verildi):** 4 ara düğüm var. $2 \cdot 4 + 1 = 9$ düğüm. Ağaçta 4 ara, 5 uç düğüm, toplam 9 düğüm vardır.

Soru 4 (derste sözlü, 00:45:04). "computers are an important technological tool" kelimeleriyle kurulan ikili arama ağacında "tennis" var mı? Yoksa nereye eklenir?

> **Çözüm (derste verildi):** computers, important, technological üzerinden sağa gidilir; tool'dan küçük olduğu için sola gidilmeli, ama tool yapraktır. "tennis" yoktur; eklenirse tool'un sol çocuğu olur.

Soru 5 (derste sözlü, 00:50:35). Kökü 50 olan arama ağacında tutulan sayı 93 ve 30 için hangi sorular sorulur?

> **Çözüm (derste verildi):** 93: > 50, > 75, < 95, 95'in solu boş, 93 yok. 30: < 50, > 25, < 35, 30 bulundu.

Soru 6 (derste sözlü, 01:03:52). Dengeli ağaçta (kök 6) 17 ve 2 nasıl aranır?

> **Çözüm (derste verildi):** 17: > 6, > 16, < 19, 17 bulundu. 2: < 6, < 4, < 3, 1'e gelinir, 2 yok.

Soru 7 (derste sözlü, 01:08:56). Dizide tutulan ikili arama ağacında (23, 13, 47, 11, 19, 43, 52, 7, 29) 43 nasıl aranır? 50 hangi indise eklenir?

> **Çözüm (derste verildi):** İndis 1 (23), $43 > 23$, indis 3 (47), $43 < 47$, indis 6 (43), bulundu. 50: > 23, > 47, < 52; 52 indis 7'de, 50 indis $2 \cdot 7 = 14$'e eklenir.

Soru 8 (derste sözlü, 01:28:24, 01:34:11, 01:38:53). Bölüm 8.1'deki ağacı preorder, inorder ve postorder okuyun.

> **Çözüm (derste verildi, öğrencilerle birlikte):** - Preorder: a, b, e, j, k, n, o, p, f, c, d, g, l, m, h, i. - Inorder: j, e, n, k, o, p, b, f, a, c, l, g, m, d, h, i. - Postorder: j, n, o, p, k, e, f, b, c, l, m, g, h, i, d, a.

Soru 9 (derste sözlü, 01:47:03). 25 tane a ve 2 tane z var. Her karakter 8 bitle kodlanırsa kaç bit harcanır? a 2 bitle, z 10 bitle kodlanırsa bir kazanç olur mu?

> **Çözüm (derste verildi):** 8 bitle $200+16 = 216$ bit. Değişken uzunlukla $50+20 = 70$ bit. Yarıdan daha az bit harcanır.

Soru 10 (derste sözlü, 02:05:21). Huffman kodu neden yukarıdan aşağı okunur, aşağıdan yukarı okunsa ne olur?

> **Çözüm (derste verildi):** Aşağıdan yukarı okunursa a = 1, e = 10, k = 100 olur. "k a" dizisi `1001` olur ve açıcı ilk 1'i a sanar; kalanı çözülemez. Yukarıdan aşağı kodlarda "k a" `0011` olur ve tek biçimde 001 (k), 1 (a) diye çözülür.

Soru 11 (derste sözlü, 02:41:47). BFS'de A'dan sonra B ve C yerine önce C sonra B kuyruğa konsaydı çıktı değişir miydi?

> **Çözüm (derste verildi):** Evet. Bu adımda serbestsiniz; BFS'nin tek bir doğru çıktısı yoktur.

Dersin Toparlaması

Bu derste ağaçlar konusuna giriş yapıldı. Ağaç, döngüsü olmayan ve iki düğüm arasında tek yol bulunan bir graftır. Kök, seviye, yükseklik ve aile ağacından gelen terimler (ebeveyn, çocuk, kardeş, ata, torun) tanımlandı. İkili ağaç, tam ikili ağaç ve dengeli ağaç kavramları ile tam ikili ağaçtaki düğüm sayısı ilişkileri görüldü. İkili arama ağacında kelime ve sayı aramaları yapıldı, sıralı bir diziden "önce sol" ve "terazi" kurallarıyla dengeli bir ağaç kuruldu ve ağacın $2i$, $2i+1$ kuralıyla bir dizide nasıl tutulduğu gösterildi. Ardından preorder, inorder ve postorder gezinmeleri bir örnek üzerinde öğrencilerle birlikte okundu.

Dersin ikinci yarısında Huffman kodlamasıyla bir metnin 480 bit yerine 135 bitle nasıl kodlanabildiği ve kodların neden yukarıdan aşağı okunması gerektiği anlatıldı. Kapsayan ağaç kavramı tanıtıldı; DFS yığınla, BFS kuyrukla adım adım işletildi. Son olarak köy ve su deposu örneğinde Prim algoritmasıyla toplamı 2400 olan en küçük kapsayan ağaç bulundu ve aynı algoritma Japonya haritası üzerinde görsel olarak izlendi. Hoca, bir sonraki derste en küçük kapsayan ağaç için Kruskal algoritmasıyla devam edileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.